

NAME: J. CHANDRA HASINI
ROLL_NO: 2520030086
SUB: OSSP (PRACTICAL)

1. FCFS Scheduling Algorithms

```
#include <stdio.h>
int main() {
    int n, i;
    int bt[20], wt[20], tat[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time for each process:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

 Help
 Exit

 Write Out
 Read File

 Where Is
 Replace

 Cut
 Paste

 Execute
 Justify

OUTPUT:

```
hasini@DESKTOP-UTCBO51:~$ nano fcfs.c
hasini@DESKTOP-UTCBO51:~$ gcc fcfs.c -o fcfs
hasini@DESKTOP-UTCBO51:~$ ./fcfs
```

Enter number of processes: 24

Enter burst time for each process:

P1: 2
P2: 3
P3: 8
P4: 1
P5: 7
P6: 0
P7: 3
P8: 6
P9: 1
P10: 4
P11: 2
P12: 0
P13: 2
P14: 9
P15: 7
P16: 3
P17: 6
P18: 2
P19: 0
P20: 12
P21: 7
P22: 9
P23: 73
P24: 21

Process	BT	WT	TAT
P1	2	0	2
P2	3	2	5
P3	8	5	13

P13: 2
P14: 9
P15: 7
P16: 3
P17: 6
P18: 2
P19: 0
P20: 12
P21: 7
P22: 9
P23: 73
P24: 21

Process	BT	WT	TAT
P1	2	0	2
P2	3	2	5
P3	8	5	13
P4	1	13	14
P5	7	14	21
P6	0	21	21
P7	3	21	24
P8	6	24	30
P9	1	30	31
P10	4	31	35
P11	2	35	37
P12	0	37	37
P13	2	37	39
P14	9	39	48
P15	7	48	55
P16	3	55	58
P17	6	58	64
P18	2	64	66
P19	0	66	66
P20	12	66	78
P21	0	2	2
P22	2	5	7
P23	5	13	18
P24	13	14	27

*** stack smashing detected ***: terminated
Aborted

./fcfs

hasini@DESKTOP-UTCBO51:~\$ |

2.SJF Nonpreemptive

```
#include<stdio.h>
```

```
int main() {
    int n, i, j;
    int bt[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // sort burst times
    for (int i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify

```
printf("P%d: ", i + 1);
scanf("%d", &bt[i]);
}

// sort burst times
for (int i = 0; i < n - 1; i++) {
    for (j = i + 1; j < n; j++) {
        if (bt[i] > bt[j]) {
            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;
        }
    }
}

// Waiting time
wt[0] = 0;

for (i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
}

// Turnaround time
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n",
        i + 1, bt[i], wt[i], tat[i]);
}

return 0;
}
|
```

 G Help	 O Write Out	 F Where Is	 K Cut	 T Execute
 X Exit	 R Read File	 \ Replace	 U Paste	 J Justify

OUTPUT:

```

hasini@DESKTOP-UTCBO51:~$ nano sjf.c
hasini@DESKTOP-UTCBO51:~$ gcc sjf.c -o sjf
hasini@DESKTOP-UTCBO51:~$ ./sjf
Enter number of processes: 5
Enter burst time:
P1: 2
P2: 6
P3: 9
P4: 3
P5: 1

Process BT      WT      TAT
P1      1      0      1
P2      2      1      3
P3      3      3      6
P4      6      6     12
P5      9     12     21
hasini@DESKTOP-UTCBO51:~$

```

3. Priority

```

#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time and priority:\n");

    for (i = 0; i < n; i++) {
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);

        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }

    // Sort according to priority
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (priority[i] > priority[j]) {
                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    }
}

```

Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was

```

        for (j = i + 1; j < n; j++) {
            if (priority[i] > priority[j]) {
                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tPriority\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\n",
               i + 1, bt[i], priority[i], wt[i], tat[i]);
    }

    return 0;
}

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/_ Go To Line M-E Redo M-G Copy ^B Where Was

OUTPUT:

```

hasini@DESKTOP-UTCBO51:~$ nano Priority.c
hasini@DESKTOP-UTCBO51:~$ nano Priority.c -o Priority.c
Invalid operating directory: Priority.c
hasini@DESKTOP-UTCBO51:~$ nano Priority.c -o Priority
Invalid operating directory: Priority
hasini@DESKTOP-UTCBO51:~$ gcc Priority.c -o Priority
hasini@DESKTOP-UTCBO51:~$ ./Priority
Enter number of processes: 6
Enter burst time and priority:
P1 Burst Time: 2
P1 Priority: 5
P2 Burst Time: 7
P2 Priority: 9
P3 Burst Time: 1
P3 Priority: 7
P4 Burst Time: 3
P4 Priority: 2
P5 Burst Time: 9
P5 Priority: 2
P6 Burst Time: 0
P6 Priority: 5

Process BT      Priority      WT      TAT
P1      3          2           0        3
P2      9          2           3       12
P3      2          5          12       14
P4      0          5          14       14
P5      1          7          14       15
P6      7          9          15       22
hasini@DESKTOP-UTCBO51:~$ |

```

4. ROUND ROBIN

```
GNU nano 8.7.1 roundrobin.c *
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    // Round Robin
    while (1) {
        done = 1;

        for (i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0;

                if (rem_bt[i] > quantum) {
                    time = time + quantum;
                    rem_bt[i] = rem_bt[i] - quantum;
                }
                else {
                    time = time + rem_bt[i];
                    wt[i] = time - bt[i];
                    rem_bt[i] = 0;
                }
            }
        }

        if (done == 1)
            break;
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo M-6 Copy

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo M-6 Copy M-B Where Was

```
GNU nano 2.7.1 roundrobin.c *
    if (rem_bt[i] > 0) {
        done = 0;

        if (rem_bt[i] > quantum) {
            time = time + quantum;
            rem_bt[i] = rem_bt[i] - quantum;
        }
        else {
            time = time + rem_bt[i];
            wt[i] = time - bt[i];
            rem_bt[i] = 0;
        }
    }

    if (done == 1)
        break;
}

// Turnaround time
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n",
        i + 1, bt[i], wt[i], tat[i]);
}

return 0;
}
```

OUTPUT:

```
hasini@DESKTOP-UTCBO51:~$ nano roundrobin.c
hasini@DESKTOP-UTCBO51:~$ gcc roundrobin.c -o roundrobin
hasini@DESKTOP-UTCBO51:~$ ./roundrobin
Enter number of processes: 4
Enter burst time:
P1: 2
P2: 8
P3: 5
P4: 3
Enter time quantum: 3

Process BT      WT      TAT
P1      2       0       2
P2      8      10      18
P3      5      11      16
P4      3       8      11
hasini@DESKTOP-UTCBO51:~$ |
```